

Software Architecture

Exercise – Domain Driven Design

BSc



Ingo Arnold

Copyright Notice

© Ingo Arnold. All rights reserved.

You may use these materials for your personal internal reference purposes only.

You may not reuse these materials in creating your own educational offerings or in your own educational situations like courses, lectures, or seminars nor may you distribute, transfer, resell or otherwise make them available to any other person or party without the expressed permission of the author.

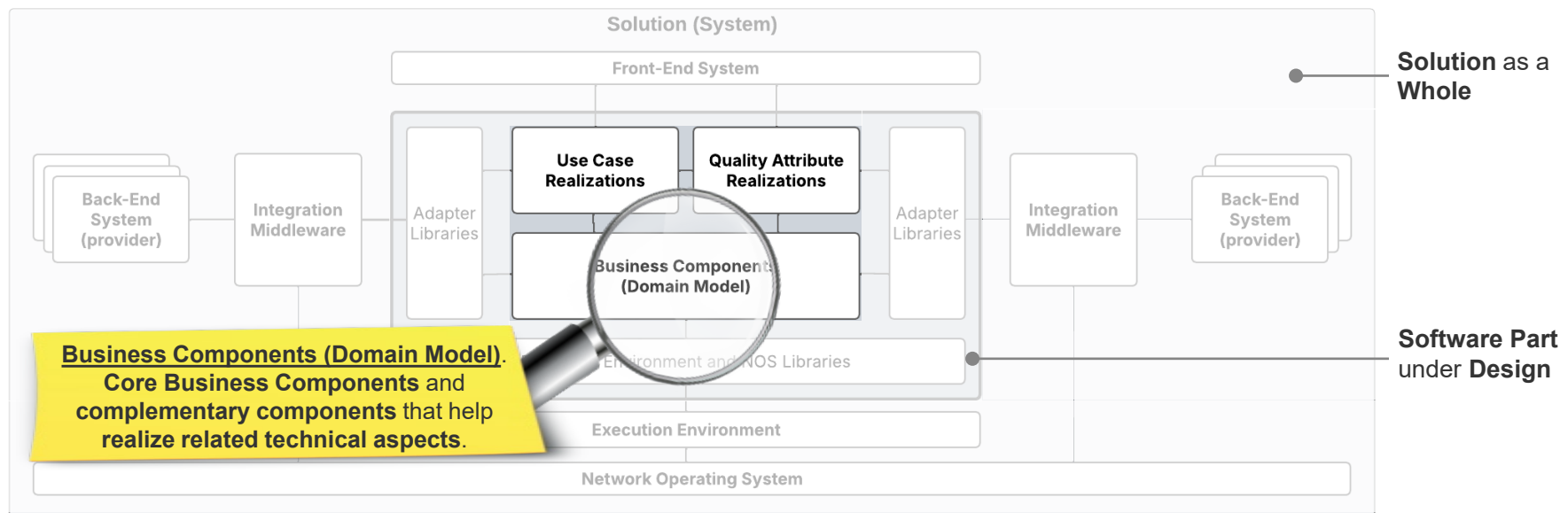
URLs or other references to Internet Web sites in the training materials are subject to change without notice. Unless otherwise indicated, all companies, organisations, domain names, email addresses, persons, places and events depicted in the Training Materials are for illustrative purposes only and are fictitious. No real connection is intended or inferred.

These training materials are provided “as is” and the author makes no warranties, express or implied, with respect to these training materials.

Exercise Opening

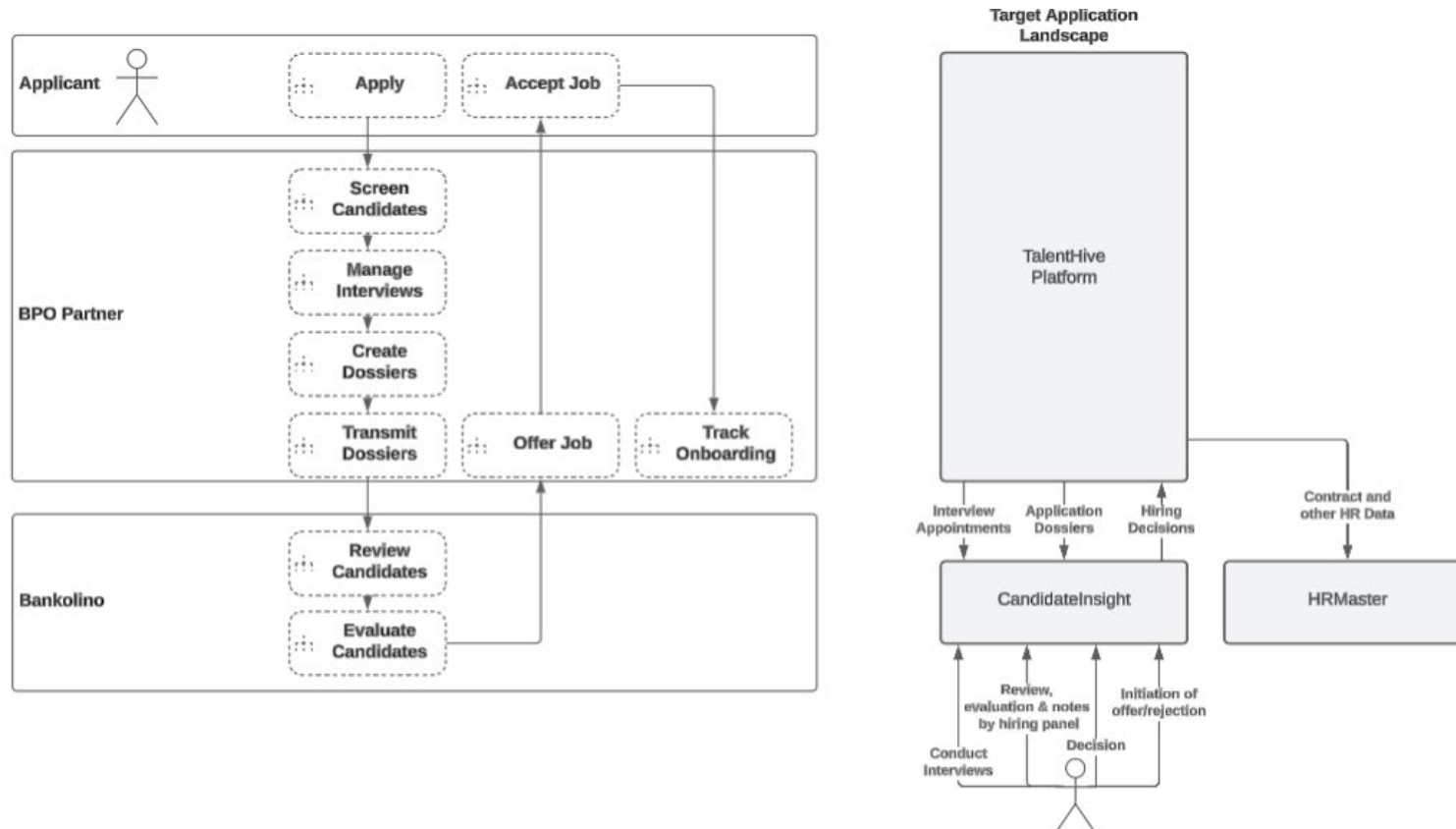
Architecture Context

In today's exercise, we will pursue **Domain-Driven Design**, with a **special focus** on addressing the **core Domain Model** and respective **Business Components**.



Exercise Opening Overview

In today's **exercise**, we **continue** what we **started last week (Business Development)**, where we arrived at the following.



Exercise Opening

Overview

In today's **exercise**, we **continue** what we **started last week (Business Development)**, where we arrived at the following.

CandidateInsight

Key Capabilities

Receipt and presentation of applicant dossiers

- Automated import via API from the BPO partner.
- Visualisation of structured applicant information and documents (CV, evaluations, screening results).

Evaluation and commenting by the hiring panel

- Role-based access control for panel members.
- Evaluation matrix (e.g. score, traffic light rating, text comments) Option for discussion and collaborative commenting.

Decision-making

- Consolidation of individual evaluations.
- Recording of the final hiring decision.
- Transfer of the decision to the HR system or to the BPO for further processing

Audit & compliance

- Logging of all evaluations, comments and decisions.
- Traceability of decision-making processes (auditability).

Security

- Access only for authorised employees.
- Encryption of sensitive personal data.
- GDPR-compliant data processing.

Exercise Opening

Overview

Today, we will use a **different approach** (different **from** the **scenario-driven design** we've **experienced already**) to **arrive** at an **initial software design: Domain-Driven Design**.

In addition to a **case-specific domain model**, we will complete the **DDD transition** and **derive** an initial version of a **class model** so that we already have a concrete basis for the new application.

We will also **discuss architecture principles** as well as *how*, *why* and *where* they should be used in our design.

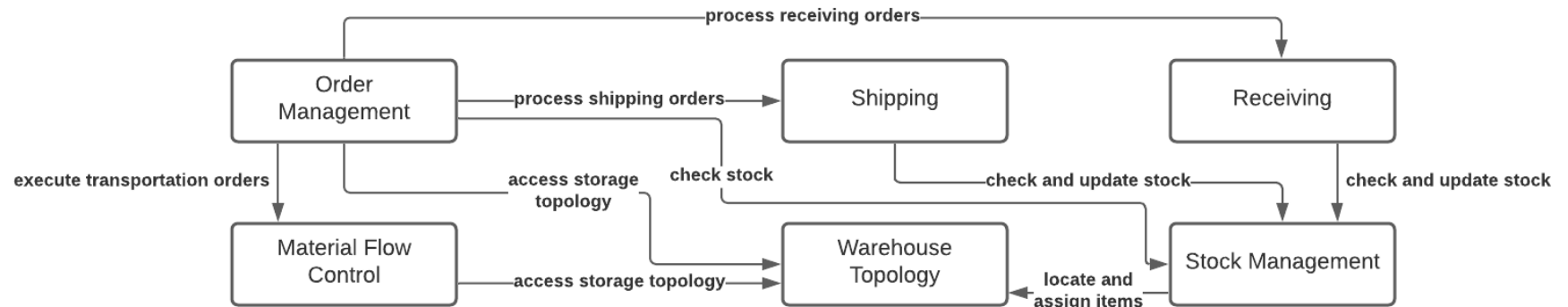
Domain-Driven Design

Domain Model

A **Domain Model** establishes a **common language** to **consistently name** and understand **key terms**, **concepts** and their **relationships** in a given **business domain**.

Rather, it is a **living working tool** that is meant to **increase efficiency** and **effectiveness** of **communication** across a broad **variety of stakeholders** and to **evolve iteratively** as a **journey progresses**.

Illustrative example of a simplified *Domain Model* for warehouse management:



Domain-Driven Design

Domain Model

A **Domain Model** is not a **technical data model** or **solution design** (despite being mis-concepted as such sometimes).

A **Domain Model** does not **define any system architecture** or **implementation** and does not **claim** to be **final and complete** at any given point in time.

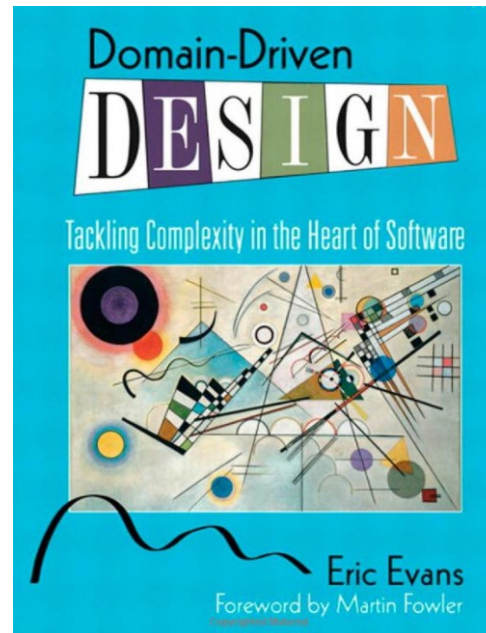
The **aim** of a **Domain Model** is to **detect** and **avoid misunderstandings**, facilitate communication and create a **Ubiquitous Language** for sound technical and business decisions – especially in interdisciplinary teams with different perspectives on the domain.

Domain-Driven Design

Domain Model

A **Domain Model** provides the **initial step** in **transforming requirements** into a **sustainable software architecture** and implementation.

In **Eric Evans** book **Domain-Driven Design** a **Domain Model** is a **first** (and yet technology-agnostic) **step** in **developing a sound software design**.



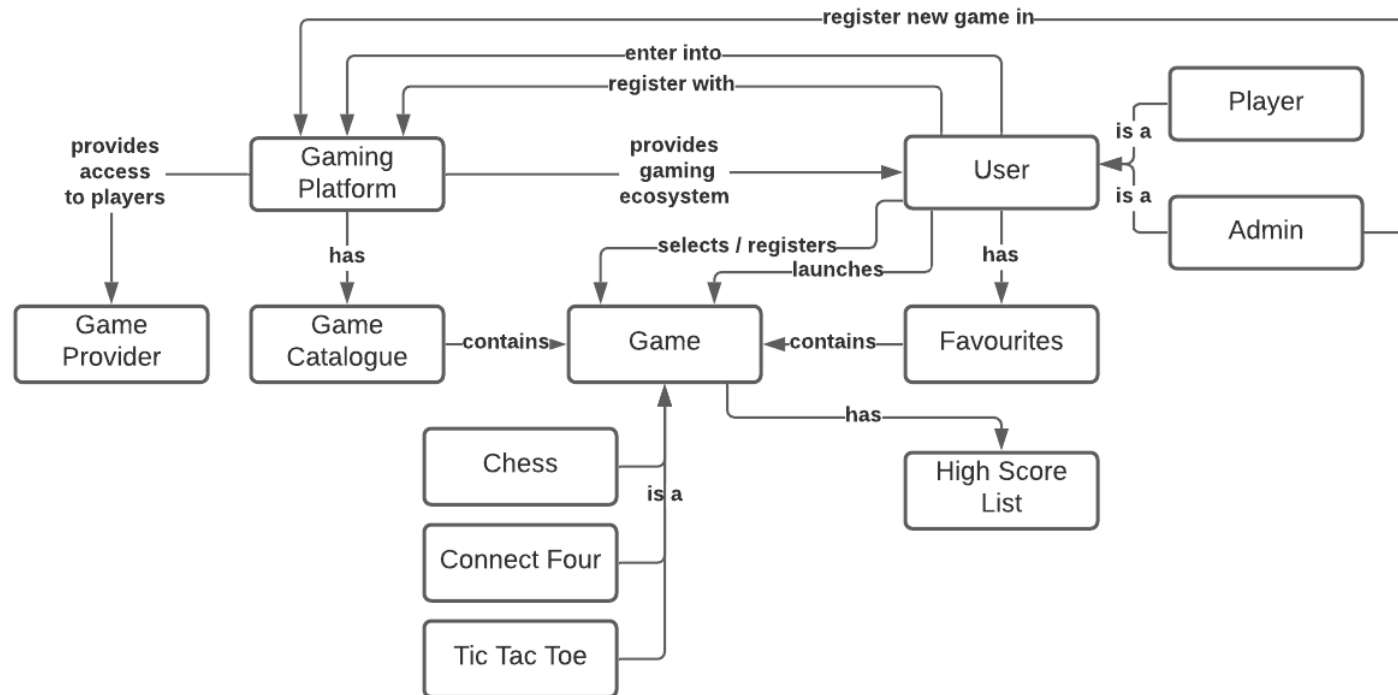


example

Domain-Driven Design

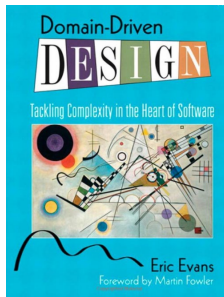
Domain Model

We could e.g. have created a **Domain Model** for the **gaming platform**.



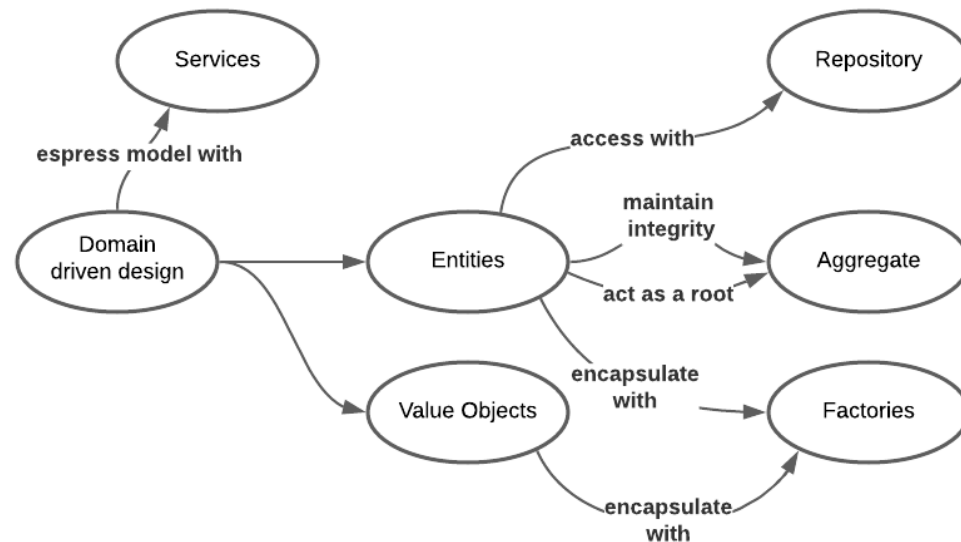
Domain-Driven Design

Domain Model – Domain-driven Design



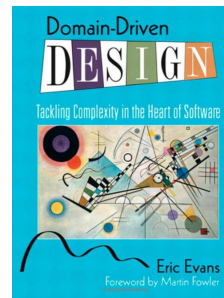
Domain-driven design distinguishes the following components of the *Domain Model*:

- Entities (reference objects)
- Value objects
- Aggregates
- Associations
- Service objects (services)
- Domain events
- Modules (modules, packages)
- Factories
- Repositories



Domain-Driven Design

Domain Model – Domain-driven Design



Domain-driven design distinguishes the following components of the *Domain Model*:

- **Entities (reference objects)**

Objects of the model, which are not defined by their properties, but by their identity. For example, a person is usually represented as an entity. Thus, a person remains the same person if his or her properties change; and he or she is different from another person even if the latter has the same properties. Entities are often modelled using unique identifiers.

- **Value objects**

Objects of the model that do not have or require a conceptual identity and are thus defined solely by their properties. Value objects are usually modelled as immutable objects, which makes them reusable (cf. Flyweight) and distributable.

- **Service objects (services)**

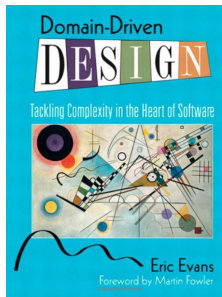
In domain-driven design, functionalities that represent an important concept of the domain model and conceptually belong to several objects of the domain model are modelled as independent service objects. Service objects are usually stateless and therefore reusable classes without associations, with methods that correspond to the provided functionalities. These methods are passed the value objects and entities necessary to process the functionality.

- **Aggregates**

Aggregates are combinations of entities and value objects and their associations to a common transactional unit. Aggregates define exactly one entity as the only access to the entire aggregate. All other entities and value objects may not be statically referenced from outside. This guarantees that all invariants of the aggregate and the individual components of the aggregate can be ensured.

Domain-Driven Design

Domain Model – Domain-driven Design



Domain-driven design distinguishes the following components of the *Domain Model*:

- **Factories**

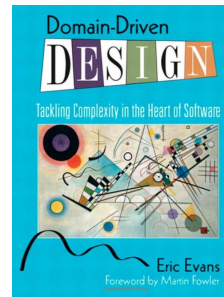
Factories are used to outsource the creation of domain objects to special factory objects. This is useful if either the creation is complex (and, for example, requires associations that the subject object itself no longer needs) or the specific creation of the subject objects should be able to be exchanged at runtime. Factories are usually implemented by generating design patterns such as abstract factory, factory method or builder.

- **Repositories**

Repositories abstract the persistence and search of business objects. Repositories separate the technical infrastructure and all access mechanisms to it from the business logic layer. For all business objects that are loaded via the infrastructure layer, a repository class is provided that encapsulates the loading and search technologies used from the outside. The repositories themselves are part of the domain model and thus part of the business logic layer. They are the only ones that access the objects of the infrastructure layer, which are usually implemented using the design patterns Data Access Objects, Query Objects or Metadata Mapping Layers.

Domain-Driven Design

Domain Model – Domain-driven Design



Domain-driven design distinguishes the following components of the *Domain Model*:

- **Domain events**

Domain events are objects that describe complex, possibly dynamically changing, actions of the domain model that cause one or more actions or changes in the domain objects. Domain events also enable the modelling of distributed systems. The individual subsystems communicate exclusively via domain-oriented events, so they are strongly decoupled and the entire system is thus more maintainable and scalable.[9].

- **Modules (modules, packages)**

Modules divide the domain model into functional (not technical) components. They are characterized by strong internal cohesion and low coupling between the modules.

- **Associations**

Associations, as defined in UML, are relationships between two or more objects of the domain model. Here, not only static relationships defined by references are considered, but also dynamic relationships that arise, for example, only through the processing of SQL queries.

Case Study

Develop a Domain Model

Form groups in which you will work on the following tasks.

The task is to **develop a Domain Model focusing** on the **area** primarily **covered** by the **CandidateInsight application**.

Identify and note **key domain concepts** and the **relationships** between them (in the style of a concept map as in the **gaming platform example**).

Start by noting down **key conceptual terms**. You will see that **a concept** you **start with** will **lead** you to **other concepts through association**.

Always **note** the **type of relationship** (association) **between concepts**.

Check whether **synonyms** exist for certain terms you note down to describe a concept and note these down accordingly.

Do **not spend** too much **time discussing whether** a **concept** is **significant enough**; just note it down.

Initially identified **concepts** can **always be sorted out later**. For the first run, it makes more sense to **cover a broad range** and **not go into too much depth** yet.



example

Case Study

Develop a Domain Model

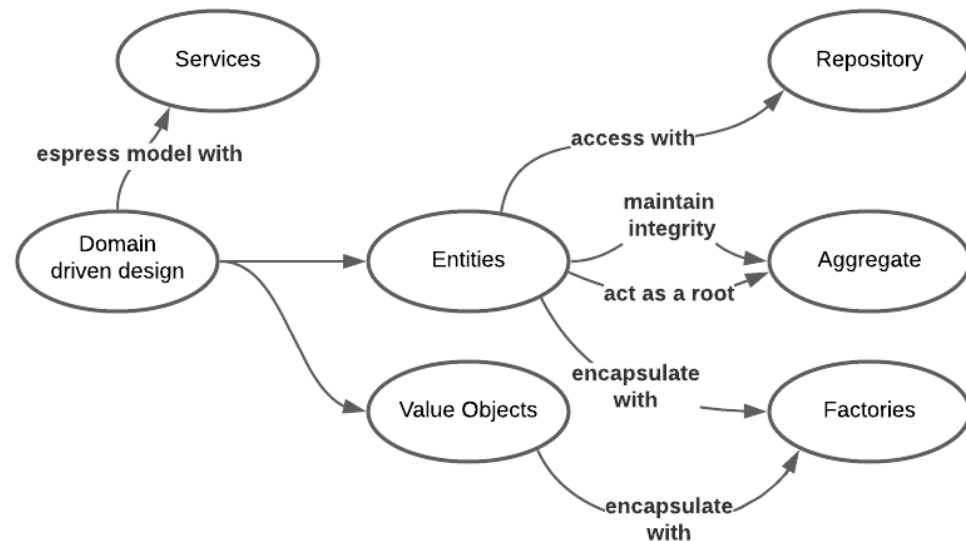
Here I will share my reference solution with you.

Case Study

Enrich your Domain Model by DDD-Concepts

Take your **Domain Model** and **consider** which of the **following DDD stereotypes** corresponds to each domain concept:

- Entities (Reference Objects)
- Value Objects
- Aggregates
- Service Objects (Services)
- Factories
- Repositories





example

Case Study

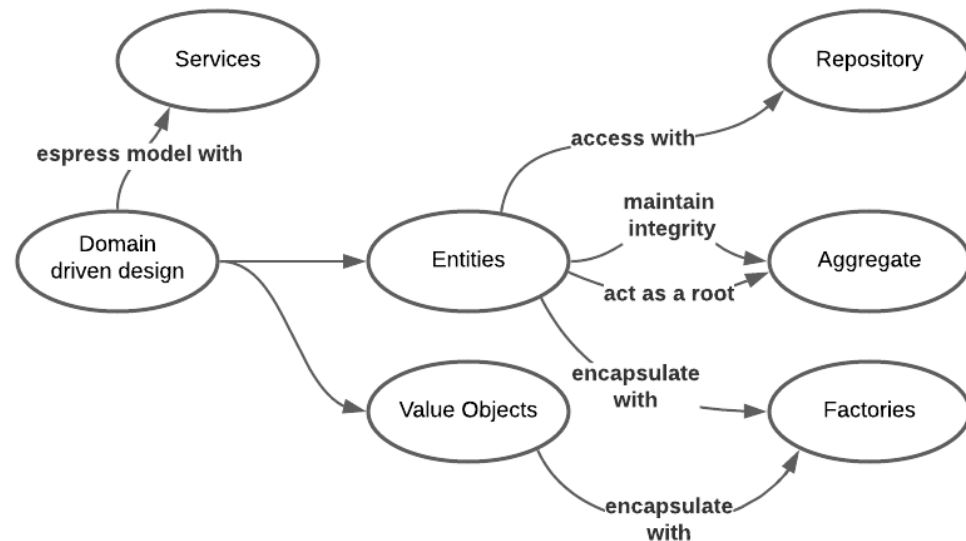
Enrich your Domain Model by DDD-Concepts

Here I will share my reference solution with you.

Case Study

Enrich your Domain Model by DDD-Concepts

Next, **complement** your **class diagram by classes** which are **required** by **DDD** but **not yet reflected** in your **domain model** – i.e., add classes for Services, Factories, Repository.





example

Case Study

Enrich your Domain Model by DDD-Concepts

Here I will share my reference solution with you.

Case Study

Enrich your Domain Model by DDD-Concepts

In the **next step**, you **determine suitable methods** for the identified **classes** and add them accordingly.

We are **not interested** in the **domain-specific methods** of the respective classes, but **rather** in the **DDD-typical methods** for each of the respective DDD categories or DDD types (e.g., service, entity, value object).

For example, **classes** of the **repository type** **require functions** for **saving** and **persisting** e.g., an **Entity object**, as well as functions for **recreating/restoring** an **Entity** object from its **persistence store**.

Now it's **your turn** – **please select** at **least one class** for **each DDD type** and **add appropriate methods** to it.



example

Case Study

Enrich your Domain Model by DDD-Concepts

Here I will share my reference solution with you.

Case Study

Discuss Architecture Principles for your Design

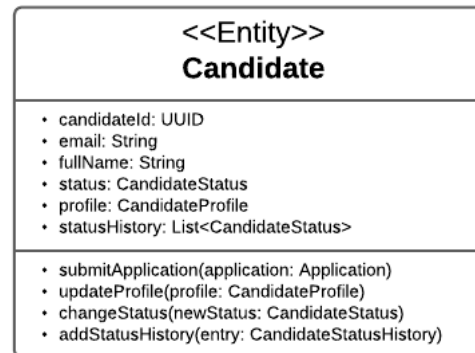
In the **next step** we will **enhance** our **design** further by **reflecting** and **applying** **architecture principles** for **selected classes**.

Case Study

Discuss Architecture Principles for your Design

Discuss the **Single Responsibility Principle (SRP)** using my *Candidate* class:

- Does ***Candidate*** have **too many responsibilities**?
- What would you **suggest** to **improve** the **situation** and **address the SRP** more appropriately?
- If moving ***CandidateStatus*** into its own module, **how would the *State Pattern* support** you here?





example

Case Study

Discuss Architecture Principles for your Design

Here I will share my reference solution with you.

Case Study

Discuss Architecture Principles for your Design

Discuss the **Single Responsibility Principle (SRP)** using my *Candidate* class:

- Does **Candidate** have **too many responsibilities**?
- What would you **suggest to improve the situation** and **address the SRP** more appropriately?
- If moving **CandidateStatus** into its own module, **how would the State Pattern** support you here?

How would **adapting the State Pattern** better **cope** with the given **situation**?

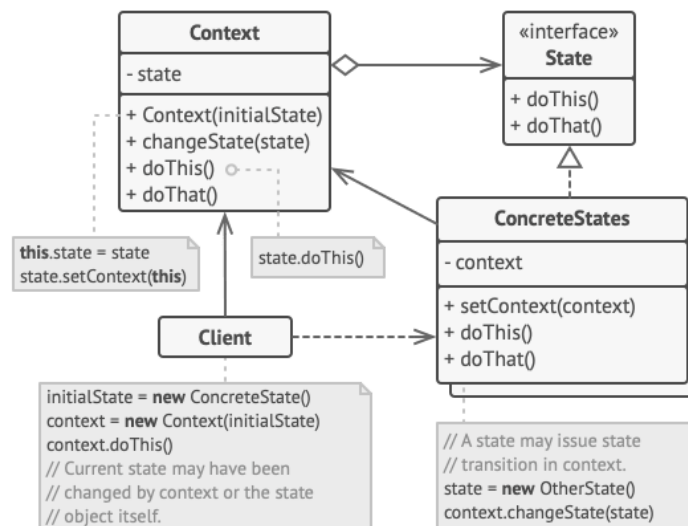
1 **Context** stores a reference to one of the concrete state objects and delegates to it all state-specific work. The context communicates with the state object via the state interface. The context exposes a setter for passing it a new state object.

2 The **State** interface declares the state-specific methods. These methods should make sense for all concrete states because you don't want some of your states to have useless methods that will never be called.

3 **Concrete States** provide their own implementations for the state-specific methods. To avoid duplication of similar code across multiple states, you may provide intermediate abstract classes that encapsulate some common behavior.

State objects may store a backreference to the context object. Through this reference, the state can fetch any required info from the context object, as well as initiate state transitions.

4 Both context and concrete states can set the next state of the context and perform the actual state transition by replacing the state object linked to the context.





example

Case Study

Discuss Architecture Principles for your Design

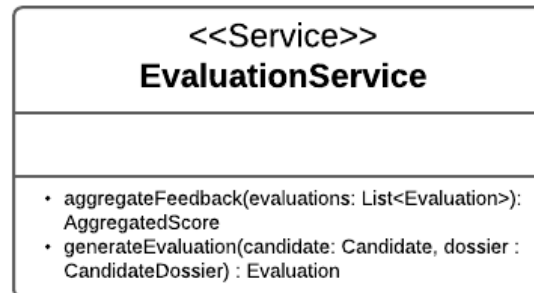
Here I will share my reference solution with you.

Case Study

Discuss Architecture Principles for your Design

Discuss the **Open Closed Principle (OCP)** referring my *EvaluationService* class:

- In my approach, *EvaluationService* implements evaluation logic directly, which means that new evaluation criteria require code modification.
- How can we make the evaluation logic extensible without changing existing classes?





example

Case Study

Discuss Architecture Principles for your Design

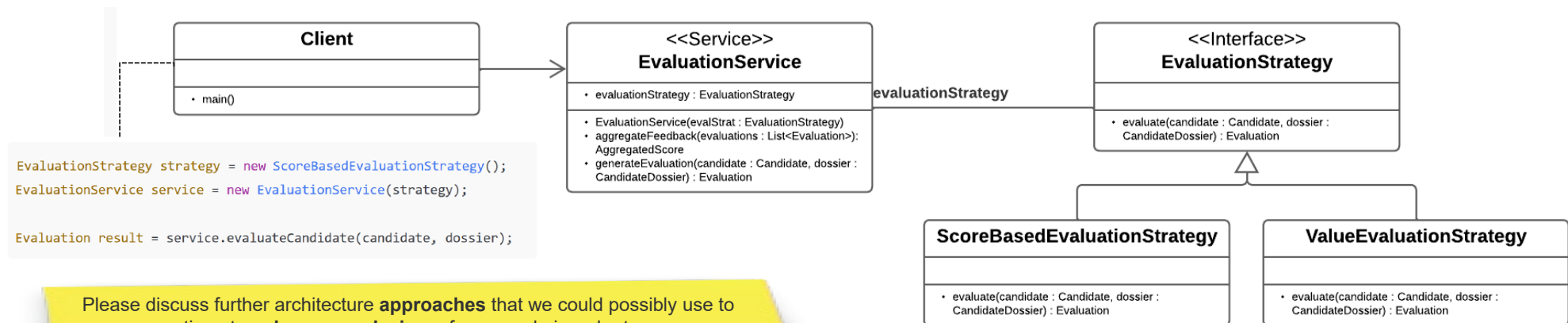
Here I will share my reference solution with you.

Case Study

Discuss Architecture Principles for your Design

Discuss the **Open Closed Principle (OCP)** referring my *EvaluationService* class:

- In my approach, *EvaluationService* implements evaluation logic directly, which means that new evaluation criteria require code modification.
- How can we make the evaluation logic extensible without changing existing classes?



Please discuss further architecture **approaches** that we could possibly use to continue to **enhance** our **design** – for example in order to ...

- more flexibly **select strategies** at **runtime**.
- flexibly **extend evaluations** with e.g., **logging** or **security**.
- **execute multiple strategies** in a combined fashion.



example

Case Study

Discuss Architecture Principles for your Design

Here I will share my reference solution with you.

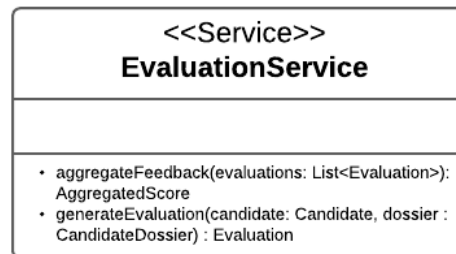
Case Study

Discuss Architecture Principles for your Design

Discuss the **Open Closed Principle (OCP)** referring my *EvaluationService* class:

- How could we **use Inheritance (rather than delegation)** as an **approach to address the Open Closed Principle**?

How could we **use Inheritance (rather than delegation)** as an **approach to address the Open Closed Principle**?





example

Case Study

Discuss Architecture Principles for your Design

Here I will share my reference solution with you.

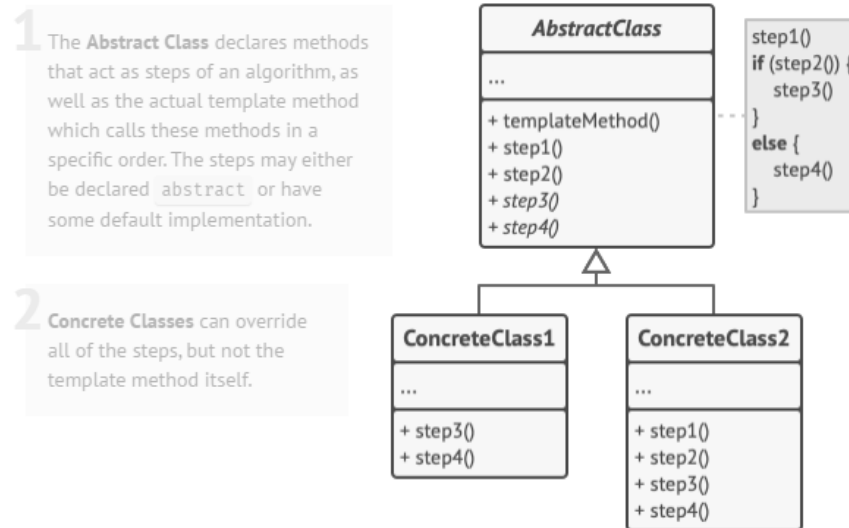
Case Study

Discuss Architecture Principles for your Design

Discuss the **Open Closed Principle (OCP)** referring my *EvaluationService* class:

- How does the **Template Method design pattern** work, and how could it further improve our design?

How does the **Template Method design pattern** work, and how could it further improve our design?





example

Case Study

Discuss Architecture Principles for your Design

Here I will share my reference solution with you.

Case Study

Discuss Architecture Principles for your Design

Overcome too tight coupling and respective dependencies:

- The entire **CandidateInsight system** must **send notifications** – for example ...
 - to the **hiring panel**: “New applicant available for evaluation”.
 - to **HR**: “Evaluation completed”.
 - to **applicants**: “Status updated”.
- In the **current design**, the **CandidateInsight** system **depends directly** on a **specific email / message implementation**.
- **Which (a) principle** is being **violated** here, and **(b) what changes** to the **current design** could **overcome this violation**?





example

Case Study

Discuss Architecture Principles for your Design

Here I will share my reference solution with you.

Questions

